

LLM Applications

生成式人工智能應用模式

Data Generation

Synthetic RAG

Textbook Data

Coding

Prompt Function

依據 Prompt Engineering Guide Applications 章節整理，加入警務與行政應用例子。

本節學習目標

完成本節後，學員應能：

- 說明 LLM 不只可用於聊天，也可用於生成資料、建立工作流、輔助編程與分類
- 設計可控的 synthetic data prompt，並知道如何檢查品質
- 理解 synthetic RAG 資料如何支援檢索模型訓練與評測
- 使用 prompt function 建立可重用的日常工作模板
- 辨識 LLM 應用中的風險：資料偏差、幻覺、格式錯誤、未測試程式碼

六個應用模式

Data

生成分類、摘要、測試樣本

RAG

生成 query-document 訓練資料

Textbook

生成高品質教材式資料

Coding

生成、補全、解釋程式與 SQL

Case Study

用實驗比較 prompt 改動效果

Function

把 prompt 封裝成可重用工具

本節重點不是背 prompt，而是學會把 LLM 應用設計成可測試、可重用、可覆核的流程。

Topic 1 | Generating Data

用 LLM 快速產生實驗、訓練與評測樣本

Prompting Guide 的基本例子：要求模型產生情感分類樣本，例如 8 個 positive、2 個 negative，並以固定格式輸出。

Data Generation 的用途

LLM 可快速生成：

- 分類樣本
- 摘要練習材料
- 測試案例
- 問答對
- 錯誤範例
- rubric 評分樣本

警務與行政例子

- 生成報案摘要訓練樣本
- 生成不同語氣的投訴信
- 生成資料缺漏案例
- 生成公文改寫練習
- 生成 SOP 問答對

基本 Prompt：生成分類資料

任務：產生 12 個「投訴信分類」練習樣本。

分類標籤：

- 服務態度
- 程序延誤
- 執法爭議
- 資料查詢

要求：

- 每個標籤至少 2 個樣本
- 每個樣本 1-2 句
- 不使用真實個人資料
- 輸出欄位：id, text, label, reason

合成資料可用於教學和初步測試，但不能直接代表真實分佈；正式模型仍需真實資料評估。

Synthetic Data 的品質控制

風險	控制方法
太重複	加入變量、場景、角色、語氣、長度要求
標籤錯誤	要求輸出 reason，再抽樣人工檢查
過度理想化	明確要求包含模糊、缺漏、矛盾樣本
敏感資料	明確禁止真實姓名、電話、地址、案號
無法評估	保留 seed、prompt、版本、生成時間

Example：生成「資料缺漏」練習樣本

請生成 8 個警務文件處理練習樣本。

每個樣本是一段 2-3 句案件摘要，但必須刻意缺少一類重要資料。

缺漏類型：

1. 時間不明
2. 地點不明
3. 涉事人身份不明
4. 事件經過不完整

輸出格式：

```
| id | case_summary | missing_type | why_it_matters | follow_up_question |
```

限制：

- 不使用真實姓名、電話、地址、案號
- case_summary 要像真實文書，但不可涉及真實個案

這類資料適合訓練學員做「查漏」和「追問問題」，而不是訓練最終決策。

Example Output：資料缺漏樣本

id	case_summary	missing_type	follow_up_question
01	報案人稱其手機於商場內遺失，懷疑被人取走，現要求警方協助處理。	時間不明	手機大約在何時最後一次見到？
02	報案人表示曾與一名男子發生爭執，其後發現車身有刮痕。	涉事人身份不明	該男子是否可描述外貌或身份資料？
03	投訴人稱已多次提交資料，但一直未收到回覆。	事件經過不完整	分別於何日、透過何方式提交資料？

教學時可讓學員判斷：這些樣本缺什麼？應如何追問？哪些地方不能自行推測？

Example：生成正反例配對

請生成 6 組警務摘要寫作「正反例」。

每組包含：

1. bad_example: 常見錯誤寫法
2. issue: 錯在哪裏
3. improved_example: 較佳寫法
4. teaching_point: 一句教學重點

錯誤類型必須平均覆蓋：

- 把推測寫成事實
- 沒有標明資料來源
- 缺少時間地點
- 語氣不夠中立
- 混合多件事但無分段
- 未列出待核實事項

這類資料適合做 pair discussion：先讓學生找錯，再比較 LLM 給出的 improved version。

Example：生成 Rubric 評分表

請為「案件摘要寫作」設計一份 5 分制評分 rubric。

評分面向：

- 事實完整性
- 來源標示
- 中立語氣
- 待核實事項
- 結構清晰

每個面向請提供：

1 分、3 分、5 分的具體描述。

最後生成 3 個學生練習題，每題附標準答案要點。

教學技巧：先讓 LLM 生成 rubric，再由老師和學生一起修改 rubric，討論「哪些標準應更嚴格」。

Example：生成多語版本資料

請生成 5 條投訴摘要樣本，每條包含：

- zh_hant：繁體中文
- pt：葡文
- en：英文
- label：投訴類別
- note：翻譯時容易失真的地方

投訴類別：

服務態度、程序延誤、資料查詢、設施安排

限制：

- 不使用真實個人資料
- 三種語言語義要一致
- note 要指出關鍵詞，例如「delay」「follow-up」「態度」

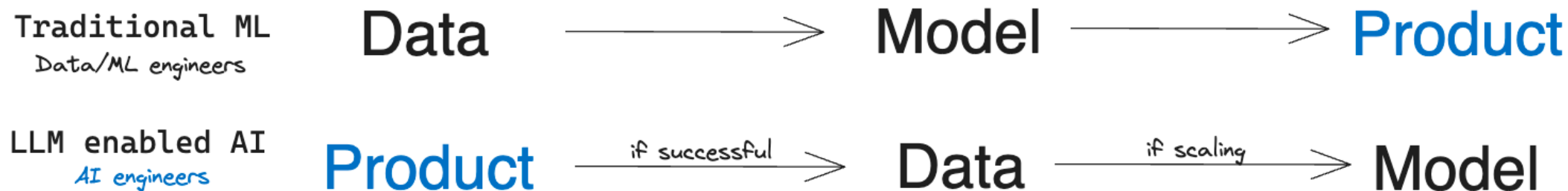
可用來教學生：多語資料生成後，需要核對語義一致性，不可只看格式。

Topic 2 | Synthetic Dataset for RAG

沒有標註資料時，如何生成檢索訓練樣本

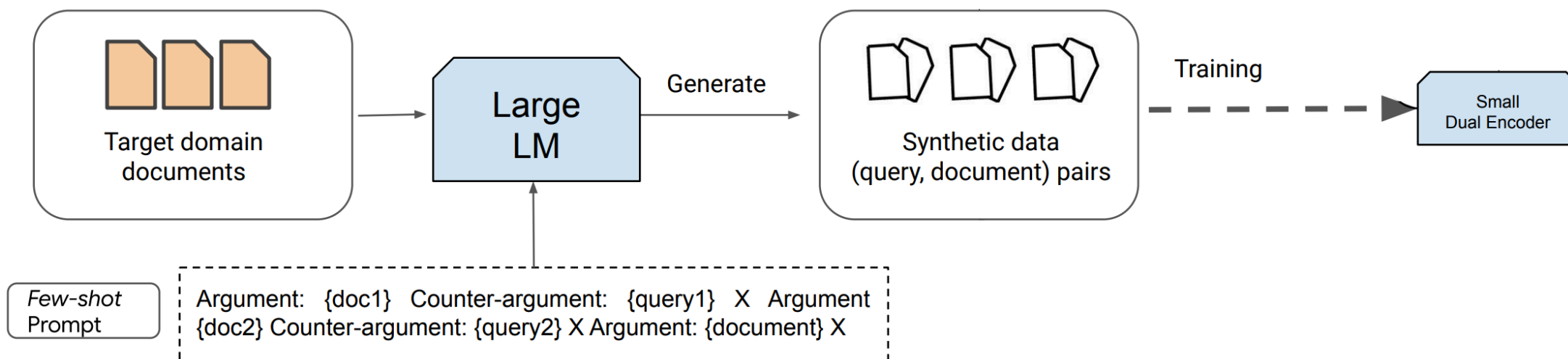
RAG 系統的品质很大程度取決於 retriever。當特定語言或專門領域缺少 query-document 標註資料時，可以用 LLM 產生合成查詢。

AI 產品開發流程的轉變



Prompting Guide synthetic RAG：LLM 讓團隊可以先快速驗證想法，再進入傳統資料收集與訓練流程。

Synthetic RAG 的基本流程



用少量人工標註例子 + 大量未標註文件，讓 LLM 為文件生成 query，形成 query-document pair。

RAG 合成資料：警務例子

假設有一批內部 SOP 或公開法例文本，但沒有查詢標註資料。

任務：根據以下程序條文，產生一個警務人員可能會問的查詢。

查詢要求：

- 使用繁體中文
- 不超過 25 字
- 必須能檢索到該段文字
- 不要直接複製條文句子

文件段落：

{{貼入 SOP 或法例段落}}

輸出：

query:

reason:

Example：SOP 段落生成 Query

文件段落

如報案資料不足以確認事件發生時間、地點或涉事人身份，
承辦人員應先記錄已知事實，並向報案人補充查詢。
補充資料未完成前，不應將未核實內容寫成確定事實。

生成結果

query:
報案資料不足時應如何補充查詢

reason:
此查詢對應「資料不足」「補充查詢」「不得寫成確定事實」三個程序重點。

這就是 query-document pair，可用於 RAG retriever 的評測或微調。

Example：同一段文件，不同 Intent

Retrieval intent	合成 query
找程序步驟	報案資料不足下一步怎樣處理
找禁止事項	未核實內容可否寫成確定事實
找補充問題	事件時間地點身份不明如何追問
找文書要求	已知事實和未核實內容如何區分

同一段文件可生成多個 query，但每個 query 都要標明 intent，否則評測時會混淆。

Example：生成 Negative Queries

RAG 評測不只需要「應該檢索到」的 query，也需要「不應檢索到」的 query。

根據以下 SOP 段落，生成：

1. `positive_query`：應能檢索到此段
2. `hard_negative_query`：語義相近但不應檢索到此段
3. `reason`：為何 `hard negative` 不相關

SOP 段落：

{{貼入段落}}

輸出格式：

| `positive_query` | `hard_negative_query` | `reason` |

Hard negative 很重要：它能測試 retriever 是否只是看到相似字詞就錯誤命中。

Example : RAG 評測集格式

```
[
  {
    "query_id": "q001",
    "query": "報案資料不足時應如何補充查詢",
    "relevant_doc_ids": ["sop_014"],
    "hard_negative_doc_ids": ["sop_032", "sop_041"],
    "intent": "procedure_lookup",
    "language": "zh-Hant"
  }
]
```

課堂討論：

- `relevant_doc_ids` 是否需要多於一個？
- `hard_negative_doc_ids` 如何產生？
- 如果 query 太像原文，是否會高估 retriever？

Example : RAG Query 改寫練習

原始 query	問題	改寫方向
資料不足	太短、意圖不清	報案資料不足時如何補充查詢
手機案件	太泛	手機遺失懷疑被取走應記錄哪些資料
投訴	無檢索價值	投訴人稱三個月未回覆應查哪類程序
法例	過度空泛	涉及拾獲他人財物拒不交還應查哪些關鍵詞

讓學生把 vague query 改成 retrieval-ready query，是 RAG 課最實用的練習之一。

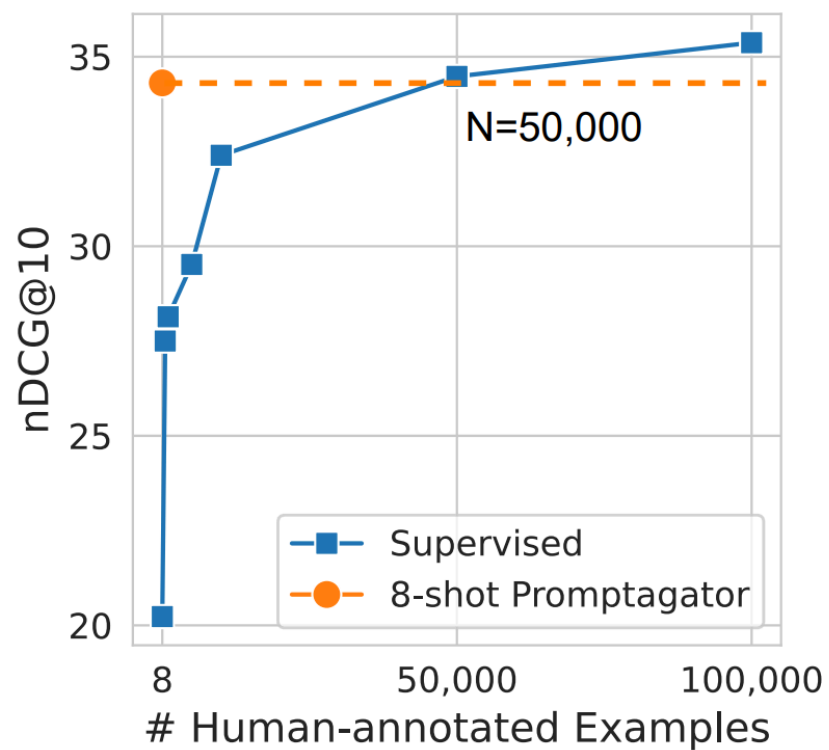
Relevance 不是固定概念

同一份文件，根據任務不同，合適 query 也不同。

任務	query 應該長什麼樣
找支持論點	問「哪段支持這個說法？」
找反駁論點	問「哪段可反駁這個說法？」
找程序要求	問「下一步需要做什麼？」
找法律依據	問「哪條法例涉及某行為？」
找表格模板	問「應使用哪份表格？」

生成 query 前，先定義 retrieval intent，否則 retriever 會學到錯誤的相關性。

合成資料 vs 人工標註



Prompting Guide 引用 PROMPTGATOR 研究：高品質合成資料可降低大量人工標註需求。

Prompt Template 也要標準化

Dataset	Prompt
ArguAna	0 Argument: passage X 1 Counter argument: query X
FiQA	0 passage X 1 query X
HotpotQA	0 Evidence: passage X 1 Vexed question: query X
DBPedia-Entity	0 entity: passage X 1 query: query X
NFCorpus	0 Article: passage X 1 Query: query X
Touché-2020	0 passage X 1 Debate: query X
TREC-Covid	0 passage X 1 Question: query X
SciFact	0 passage X 1 Finding: query X
SCIDOCS	0 passage X 1 The passage is about query X
FEVER	0 passage X 1 Is it true that query X

不同任務應有不同 prompt template；不能用同一套生成所有 retrieval data。

Topic 3 | Generating Textbooks

用「教材式資料」提升合成資料品質與多樣性

Prompting Guide 強調：生成資料最大的挑戰之一是 diversity。高品質資料應該清晰、完整、資訊充足、盡量中立。

為什麼要「Textbook-like」資料？

普通合成資料常見問題：

- 內容重複
- 語氣單一
- 說明不完整
- 缺少推理步驟
- 對新題目泛化差

教材式資料的特徵：

- 逐步解釋
- 範例清楚
- 概念集中
- 語言適合指定讀者
- 可用於訓練小模型或課堂材料

注入 Diversity：用變量控制生成

生成一段警務文書寫作教材片段。

對象：{{新入職人員 / 主管 / 文書人員}}

主題：{{案件摘要 / 投訴分類 / 補充資料要求}}

場景：{{資料不足 / 時序矛盾 / 來源不明}}

語氣：{{正式 / 教學 / 簡明}}

要求：

- 先解釋一個概念
- 再提供一個短例子
- 最後列出 3 個常見錯誤

重點是讓變量覆蓋不同受眾、場景、錯誤類型，而不是只提高 temperature。

Example：教材式輸出片段

主題：案件摘要中的「已知事實」與「待核實資料」

對象：新入職文書人員

教材片段：

案件摘要應區分已由報案人明確提供的資料，以及仍需核實的資料。

例如，「報案人稱手機於晚上約八時在商場遺失」屬於報案人陳述；
但「手機被某人盜去」若沒有證據支持，應寫成「報案人懷疑手機被取走」。

例子：

不佳寫法：嫌疑人偷走報案人手機。

較佳寫法：報案人稱手機遺失，並懷疑被他人取走，具體情況仍待核實。

常見錯誤：

1. 把懷疑寫成事實
2. 沒有標明資料來源
3. 忽略仍需補充查詢的資料

Example：分層生成教材資料

先生成「課題大綱」，再生成「教材片段」：

Step 1:

請為「警務文件查漏」生成 5 個教材課題。

每個課題包含：title, learner_level, common_error。

Step 2:

根據以下課題，生成一頁教材：

title: 區分陳述、推測與已核實事實

learner_level: 新入職人員

common_error: 把報案人懷疑寫成確定事實

輸出：概念說明、好壞例子、3 題小練習。

這種 hierarchical generation 比一次過要求「生成很多教材」更容易控制品質。

Example：生成 Micro-lesson

請生成一個 8 分鐘 micro-lesson，用於教授「如何避免把推測寫成事實」。

輸出：

1. lesson_objective: 一句
2. concept_explanation: 不超過 120 字
3. bad_example: 一句
4. corrected_example: 一句
5. quick_activity: 2 分鐘小組活動
6. check_question: 1 題選擇題，附答案

對象：非技術背景警務人員

語氣：清楚、實務、避免學術化

Example：生成不同難度練習

level	prompt 要求	教學用途
Beginner	找出摘要中缺少的人時地事物	入門查漏
Intermediate	改寫不夠中立的句子	文書品質
Advanced	比較兩份材料是否矛盾	案件整理
Supervisor	檢查草稿是否適合提交覆核	管理審批

請針對 level=Intermediate 生成 5 題練習。
每題包含：原句、問題、改寫版本、評分要點。

同一主題按 level 生成，可讓老師照顧不同背景學員。

Textbook-like Sample

To begin, let us define singular and nonsingular matrices. A matrix is said to be singular if its determinant is zero. On the other hand, a matrix is said to be nonsingular if its determinant is not zero. Now, let's explore these concepts through examples.

Example 1:

Consider the matrix $A = \text{np.array}([[1, 2], [2, 4]])$. We can check if this matrix is singular or nonsingular using the determinant function. We can define a Python function, `is_singular(A)`, which returns true if the determinant of A is zero, and false otherwise.

```
import numpy as np
def is_singular(A):
    det = np.linalg.det(A)
    if det == 0:
        return True
    else:
        return False
```

```
A = np.array([[1, 2], [2, 4]])
print(is_singular(A)) # True
```

Prompting Guide 引用 Phi-1 研究：高品質教材式資料可用於訓練較小模型。

小模型也受益於高品質資料

Date	Model	Model size	Dataset size (Tokens)	HumanEval (Pass@1)	MBPP (Pass@1)
2021 Jul	Codex-300M [CTJ ⁺ 21]	300M	100B	13.2%	-
2021 Jul	Codex-12B [CTJ ⁺ 21]	12B	100B	28.8%	-
2022 Mar	CodeGen-Mono-350M [NPH ⁺ 23]	350M	577B	12.8%	-
2022 Mar	CodeGen-Mono-16.1B [NPH ⁺ 23]	16.1B	577B	29.3%	35.3%
2022 Apr	PaLM-Coder [CND ⁺ 22]	540B	780B	35.9%	47.0%
2022 Sep	CodeGeeX [ZXZ ⁺ 23]	13B	850B	22.9%	24.4%
2022 Nov	GPT-3.5 [Ope23]	175B	N.A.	47%	-
2022 Dec	SantaCoder [ALK ⁺ 23]	1.1B	236B	14.0%	35.0%
2023 Mar	GPT-4 [Ope23]	N.A.	N.A.	67%	-
2023 Apr	Replit [Rep23]	2.7B	525B	21.9%	-
2023 Apr	Replit-Finetuned [Rep23]	2.7B	525B	30.5%	-
2023 May	CodeGen2-1B [NHX ⁺ 23]	1B	N.A.	10.3%	-
2023 May	CodeGen2-7B [NHX ⁺ 23]	7B	N.A.	19.1%	-
2023 May	StarCoder [LAZ ⁺ 23]	15.5B	1T	33.6%	52.7%
2023 May	StarCoder-Prompted [LAZ ⁺ 23]	15.5B	1T	40.8%	49.5%
2023 May	PaLM 2-S [ADF ⁺ 23]	N.A.	N.A.	37.6%	50.0%
2023 May	CodeT5+ [WLG ⁺ 23]	2B	52B	24.2%	-
2023 May	CodeT5+ [WLG ⁺ 23]	16B	52B	30.9%	-
2023 May	InstructCodeT5+ [WLG ⁺ 23]	16B	52B	35.0%	-
2023 Jun	WizardCoder [LXZ ⁺ 23]	16B	1T	57.3%	51.8%
2023 Jun	phi-1	1.3B	7B	50.6%	55.5%

資料品質和多樣性可比單純增加資料量更重要。

Topic 4 | Generating Code

LLM 作為程式、SQL、測試資料與解釋助手

Prompting Guide 的 coding 章節展示：LLM 可生成基本程式、從註解生成程式、補全函數、生成 SQL、生成 schema 和 dummy data。

Code Generation Example

Playground

Load a preset...
Save
View code
Share
...

SYSTEM
You are a helpful code assistant that can teach a junior developer how to code. Your language of choice is Python. Don't explain the code, just generate the code block itself.

USER Write code that asks the user for their name and say "Hello"

ASSISTANT

```
'''
name = input("What is your name? ")
print("Hello", name)
'''
```

+ Add message

Mode

Chat
Beta

Model

gpt-3.5-turbo

Temperature
0.7

Maximum length
256

Top P
1

Frequency penalty
0

Presence penalty
0

LLM 可依 user message 生成程式碼；system message 可控制角色、語言和輸出格式。

Coding Prompt 的基本結構

System:

你是 Python 程式助手。只輸出可執程式碼，不加入額外解釋。

User:

寫一個函數，讀取 CSV 檔案，計算每個案件類型的數量，並輸出成一個按數量降序排列的表格。

限制:

- 使用 pandas
- 函數名稱: `summarize_case_types`
- 需要處理缺失值

生成 SQL：要提供 Schema

資料表 incidents:

- IncidentId
- CaseType
- District
- ReportDate
- Status

任務:

產生 SQL，統計 2025 年每個區份的投訴數量，
只包括 Status = 'closed' 的案件。

程式碼與 SQL 必須測試。Prompting Guide 的例子提醒：模型可能漏掉 import、產生不可執行或不符合資料庫方言的語句。

Coding 的警務應用

資料清理

- CSV 欄位標準化
- 日期格式修正
- 缺失值統計

報表自動化

- 生成 SQL
- 生成 pandas script
- 生成圖表程式

學習與審查

- 解釋現有程式
- 產生測試資料
- 找出潛在錯誤

Example：生成 pandas 分析程式

你是 Python 資料分析助手。

任務：

根據 CSV 欄位生成 pandas 程式，統計每月各案件類型數量。

欄位：

- case_id
- report_date
- case_type
- district
- status

要求：

- 讀取 incidents.csv
- 將 report_date 轉為日期
- 產生 month 欄位，格式 YYYY-MM
- group by month 和 case_type
- 輸出 incidents_by_month.csv
- 補上必要 import

生成後要實際執行、檢查欄位名稱、日期格式和缺失值；不要直接把程式用在敏感原始資料。

Example：生成 SQL + 測試資料

資料表 complaints:

- complaint_id
- received_date
- category
- district
- status

請完成三件事：

1. 建立 SQLite table schema
2. 插入 10 筆 dummy data
3. 寫 SQL 查詢：統計每個 category 中 status='closed' 的數量

限制：

- dummy data 不可使用真實個人資料
- 日期集中在 2025 年
- SQL 必須可在 SQLite 執行

這個 prompt 比「幫我寫 SQL」更好，因為它同時要求 schema、測試資料和查詢，方便驗證。

Example : Debug 生成程式

以下 Python 程式執行時出錯。請幫我：

1. 說明錯誤原因
2. 提供修正版
3. 加入一個最小測試資料
4. 說明如何驗證結果

程式：

```
```python
import pandas as pd

df = pd.read_csv("incidents.csv")
df["month"] = df["report_date"].dt.strftime("%Y-%m")
print(df.groupby(["month", "case_type"]).size())
```
```

錯誤訊息：

```
AttributeError: Can only use .dt accessor with datetimelike values
```

教學重點：讓模型根據錯誤訊息修正，而不是只說「幫我 debug」。

Example：先生成分析計劃

我有一份投訴資料 CSV，想知道是否某些類別在最近三個月增加。

請不要直接寫程式。先幫我列出分析計劃：

1. 需要哪些欄位
2. 如何清理日期
3. 如何定義「最近三個月」
4. 應產生哪些表格
5. 可能有哪些誤讀風險

最後再問我是否要生成 Python 程式。

課堂可先做「分析計劃」，再生成程式；這比直接叫 LLM 寫 code 更穩。

Example：程式碼審查 Prompt

請審查以下 SQL 是否有問題。

審查角度：

- 是否符合任務
- 是否可能漏算或重複計算
- 日期條件是否明確
- 是否有資料私隱風險
- 是否需要索引或效能改善

輸出格式：

```
| issue | severity | explanation | suggested_fix |
```

SQL：

```
{{貼入 SQL}}
```

讓學生學會用 LLM 做 review，而不是只做 generation。

Topic 5 | Workplace Case Study

Prompt 改動如何影響分類效果

Prompting Guide 的 workplace case study 來自 graduate job classification：同一分類任務，prompt engineering 可令 F1 從 65.6 提升至 91.7。

Case Study 的主要發現

- 對不需要專家知識的分類任務，Few-shot CoT 不一定比 zero-shot 好
- Prompt 對 reasoning 和分類結果影響很大
- 強制嚴格模板可提高格式一致性，但可能降低分類表現
- 很小的修改也可能造成明顯差異
- 清楚指令與重複關鍵標準，是重要 performance driver

從 Case Study 學到的設計原則

| 原則 | 說明 |
|-------------|--------------------------|
| 先做 baseline | 不要假設複雜 prompt 一定更好 |
| 分開量度 | 同時看 accuracy / F1 / 格式遵從 |
| 小步實驗 | 一次只改一兩個 prompt 元素 |
| 保留錯例 | 分析錯誤比看平均分更有用 |
| 不迷信 CoT | 分類任務未必需要長推理 |

警務分類實驗設計

任務：判斷投訴摘要是否涉及「程序延誤」。

Baseline:

請判斷以下投訴是否涉及程序延誤。回答 yes/no。

Improved:

你是投訴分類助理。只根據摘要文字判斷，不可推測。

程序延誤指：承諾時限未完成、長時間未回覆、無合理原因拖延。

若資料不足，回答 unclear。

輸出格式：

label: yes/no/unclear

reason: 一句

評估表

| 測試項 | 指標 |
|-------|---------------------|
| 分類準確性 | precision、recall、F1 |
| 格式穩定 | 是否完全符合輸出格式 |
| 不確定處理 | unclear 是否合理 |
| 偏差 | 是否過度傾向 yes 或 no |
| 可解釋性 | reason 是否引用摘要內事實 |

Example：分類測試集

| id | 投訴摘要 | gold label |
|-----|-------------------------------|------------|
| A01 | 投訴人稱提交補充文件後三個月未收到任何回覆。 | yes |
| A02 | 投訴人不滿前台人員語氣生硬，但未提及處理時間。 | no |
| A03 | 投訴人表示「等了很久」，但沒有提供日期或承諾期限。 | unclear |
| A04 | 投訴人稱多次致電查詢，均被告知案件仍待處理，沒有說明原因。 | yes |

課堂可讓學員先用 baseline prompt 跑一次，再逐步加入定義、unclear 類別和輸出格式。

Example : Prompt A/B Test

| 版本 | Prompt 改動 | 預期觀察 |
|----|----------------|-----------------|
| A | 只問是否涉及程序延誤 | 容易過度判 yes |
| B | 加入程序延誤定義 | precision 應改善 |
| C | 加入 unclear 類別 | 減少硬判斷 |
| D | 要求 reason 引用原文 | 可解釋性提升 |
| E | 強制 JSON 輸出 | 格式穩定提升，但要觀察分類效果 |

Prompt engineering 應像小型實驗：保留測試集、逐步改 prompt、記錄錯誤，而不是憑感覺選一個版本。

Example：錯誤分析表

| id | gold | model | error_type | possible_prompt_fix |
|-----|---------|---------|------------|---------------------|
| A03 | unclear | yes | 過度推測 | 加入「資料不足回答 unclear」 |
| A05 | no | yes | 關鍵詞誤導 | 定義「延誤」必須有時間或承諾期限 |
| A08 | yes | unclear | 忽略隱含日期 | 要求檢查「提交後多久未回覆」 |

請根據錯誤分析表，建議下一版 prompt 應修改哪三點。
不要重寫整個 prompt，只列修改理由和預期改善。

Example：學生分組實驗

| 小組 | 任務 |
|---------|-----------------|
| Group A | 設計最短 prompt |
| Group B | 加入明確定義 |
| Group C | 加入 unclear 類別 |
| Group D | 加入 reason 和引用原文 |
| Group E | 強制 JSON 格式 |

每組用同一批 10 個測試案例，記錄：

- 分類正確數
- 格式錯誤數
- 最常見錯誤類型
- 下一版 prompt 修改建議

Example : Prompt 實驗紀錄表

```
experiment_id:  
model:  
prompt_version:  
changed_elements:  
test_set:  
accuracy:  
format_error_rate:  
notable_failures:  
teacher_notes:  
next_revision:
```

這張表可讓學生體會：prompt engineering 是可記錄、可比較的實驗，不是一次性靈感。

Topic 6 | Prompt Function

把 prompt 封裝成可重用的工作工具

Prompt Function 的想法：像函數一樣定義名稱、輸入與規則，之後用 `function_name(input)` 重複呼叫。

Prompt Function 的基本格式

```
function_name: [case_summary]
input: ["text"]
rule: [
  你是警務文件整理助理。
  根據 text 抽取人、時、地、事、物。
  不可補充未提供資料。
  輸出欄位: facts, missing_info, follow_up_questions。
]
```

之後可呼叫：

```
case_summary("{}貼入案件摘要{}")
```

Prompt Function 可以串成 Workflow

```
case_summary(raw_text)
missing_info_check(case_summary_output)
law_search_terms(missing_info_check_output)
report_draft(case_summary_output, verified_sources)
```

Prompt function 適合非工程背景學員：先把常用工作封裝成模板，再逐步建立個人或部門 prompt library。

Prompt Function vs Function Calling

| 面向 | Prompt Function | Function Calling |
|------|---------------------------------|--------------------------------------|
| 本質 | 可重用 prompt 模板 | 模型輸出工具調用參數 |
| 執行方式 | 人在 chat 中呼叫 | 系統/API 執行工具 |
| 適合對象 | 一般使用者、教學、低代碼流程 | 工程系統、agent、外部工具 |
| 風險 | 模板漂移、上下文污染 | 工具誤用、權限與日誌 |
| 警務例子 | <code>case_summary(text)</code> | <code>macau_law_search(query)</code> |

課堂練習

請設計三個 prompt functions：

| Function | 任務 |
|---|-----------------|
| <code>complaint_classifier(text)</code> | 將投訴摘要分成 3-5 類 |
| <code>missing_info_check(text)</code> | 找出資料缺口和追問問題 |
| <code>search_terms(text)</code> | 產生內部 SOP 或法例搜尋詞 |

要求：

- 每個 function 都要有 input、rule、output format
- 明確寫出不可推測、不可編造來源
- 設計一個串接 workflow

Example：三個可重用 Prompt Functions

```
function_name: [extract_facts]
input: ["text"]
rule: [
  從 text 抽取人、時、地、事、物。
  只列明文提供資料；不推測。
  輸出 JSON: people, time, location, event, objects, unknowns。
]
```

```
function_name: [risk_flags]
input: ["facts_json"]
rule: [
  根據 facts_json 找出資料缺口、矛盾和需人工覆核事項。
  不作法律結論。
]
```

```
function_name: [draft_summary]
input: ["facts_json", "risk_flags"]
rule: [
  生成正式但保守的摘要草稿。
  明確標示未核實資料和待補充問題。
]
```

Example：串接使用

```
extract_facts("
報案人稱昨晚在某商場遺失手機，懷疑被他人取走。
其後曾向保安查詢，但未能提供具體時間及手機型號。
")

risk_flags({{extract_facts 的輸出}})

draft_summary({{extract_facts 的輸出}}, {{risk_flags 的輸出}})
```

Prompt function 的輸出會成為下一步輸入；如果第一步抽取錯，後面會連鎖錯。所以要在 workflow 中加入人工檢查點。

Example：建立部門 Prompt Library

| function_name | 用途 | owner | review_cycle |
|----------------------|------------|-------|--------------|
| extract_facts | 抽取案件摘要欄位 | 文書組 | 每月 |
| complaint_classifier | 投訴初步分類 | 投訴處理組 | 每季 |
| sop_query_builder | 生成 SOP 查詢詞 | 培訓組 | 每季 |
| draft_reply | 草擬回覆信 | 行政組 | 每月 |

每個 prompt function 應保存：

- 版本
- 適用場景
- 禁止事項
- 測試案例
- 最近一次修改原因

Example : Prompt Function 版本管理

```
function_name: complaint_classifier
version: 1.2
updated_on: 2026-06-12
owner: complaint_training_team

change_log:
- v1.0: 初版, 分類服務態度 / 程序延誤 / 資料查詢
- v1.1: 加入 unclear 類別
- v1.2: 要求 reason 必須引用輸入摘要文字

known_limits:
- 短摘要可能分類不穩定
- 不處理法律責任判斷
```

這能把 prompt 從「個人技巧」變成「可管理的教學與工作資產」。

Example : Prompt Function Safety Wrapper

```
function_name: safe_draft_reply
input: ["complaint_summary", "verified_facts"]
rule: [
  根據 verified_facts 草擬回覆。
  如 complaint_summary 中有資料未出現在 verified_facts, 不得使用。
  不承認責任, 不作法律判斷, 不承諾未批准措施。
  必須加入「仍需人工覆核」欄位。

  輸出:
  draft_reply:
  used_facts:
  excluded_claims:
  needs_human_review:
]
```

安全 wrapper 的目的，是把「不能做什麼」寫入可重用模板，而不是靠每次使用者記得提醒。

整體應用地圖

| 應用 | 最適合處理 | 主要風險 |
|----------------------|-------------------------|----------------------|
| Generating Data | 快速生成練習與測試樣本 | 分佈不真實、標籤錯 |
| Synthetic RAG | 生成 query-document pairs | retrieval intent 定義錯 |
| Textbook Data | 高品質教材與訓練資料 | 成本、重複、偏差 |
| Coding | 程式、SQL、測試資料 | 未測試、缺 import、資料風險 |
| Workplace Case Study | prompt 實驗與分類評估 | 只看格式不看效果 |
| Prompt Function | 可重用工作模板 | 上下文污染、規則不清 |

本節總結

- LLM 應用不只是「問答」，也包括資料生成、檢索訓練、教材生成、編程、分類和工作流封裝
- 合成資料要重視 diversity、標籤品質和評估方法
- RAG 的合成 query 必須先定義 retrieval intent
- 程式碼生成必須測試，不應直接在敏感資料環境中執行
- Prompt function 是非工程使用者建立可重用 AI 工作流的入口
- 所有應用都需要清楚記錄 prompt、版本、輸入、輸出和人工覆核

References

- Generating Data
<https://www.promptingguide.ai/applications/generating>
- Generating Synthetic Dataset for RAG
https://www.promptingguide.ai/applications/synthetic_rag
- Tackling Generated Datasets Diversity / Textbooks
https://www.promptingguide.ai/applications/generating_textbooks
- Generating Code
<https://www.promptingguide.ai/applications/coding>
- Graduate Job Classification Case Study
https://www.promptingguide.ai/applications/workplace_casestudy
- Prompt Function
<https://www.promptingguide.ai/applications/pf>